

Real-Time Collaborative Code Editor

Software Engineering Project Proposal

Submitted by: Yuvraj Jasuja(1024030069) and Harwinder(1024030075)

Department of Computer Engineering

Thapar Institute of Engineering and Technology

August 2026

1 Higher-order Goal

The goal of this project is to make collaborative programming more accessible and interactive by providing a shared environment where multiple users can work on the same code in real time.

Programming is often a collaborative activity. Students work together on assignments, developers pair program while solving problems, and teams frequently need to discuss and modify the same codebase. However, basic collaboration often involves repeatedly sharing files, copying code through messaging platforms, or manually merging different versions of the same program.

Our project aims to reduce this friction by creating a real-time coding environment in which users can join a common room and collaborate on the same code simultaneously.

2 Introduction / Problem Statement

Collaborative coding can become inefficient when multiple people are working on the same programming task.

Consider a common classroom situation where two students are solving a programming problem together. One student writes a part of the solution and sends the file to the other. The second student modifies it and sends another version back. As the number of changes increases, it becomes difficult to keep track of the latest version of the program.

This workflow creates several problems:

- Repeated sharing of source code files.
- Multiple versions of the same program.
- Difficulty identifying the latest version.
- Delayed feedback between collaborators.
- Version conflicts when different users modify the same code.
- Lack of a simple shared environment for collaborative programming.

Existing professional collaboration tools can solve many of these problems, but they may be unnecessarily complex for small teams, students, classroom collaboration, or quick coding sessions.

The Real-Time Collaborative Code Editor addresses this problem by allowing multiple users to join a shared coding room and view code changes as they happen.

3 Objectives

The main objectives of the project are:

- Build a web-based platform for real-time collaborative code editing.
- Allow users to create or join coding rooms using a unique Room ID.
- Synchronize code changes between multiple connected users in real time.
- Display active users currently connected to a coding room.
- Provide a clean and interactive code editing environment.
- Ensure that users joining an existing room can receive the current state of the code.
- Extend the platform in future phases to support multiple programming languages and secure code execution.

4 Methodology / Solution Approach

4.1 System Architecture

The system follows a client-server architecture with real-time communication between users and the backend server.

The main components are:

- User Interface built using React.
- Code Editor implemented using CodeMirror.
- Real-time communication handled using Socket.IO.
- Backend server developed using Node.js and Express.
- Room-based communication to separate different collaborative coding sessions.

The overall architecture can be represented as:

User A $\rightarrow$ React + CodeMirror $\rightarrow$ Socket.IO $\rightarrow$ Node.js Server

The server then broadcasts relevant changes to other users connected to the same room:

Node.js Server $\rightarrow$ Socket.IO $\rightarrow$ User B, User C, ...

4.2 Real-Time Collaboration Flow

The main workflow of the system is as follows:

1. A user enters a username and creates a new coding room or enters an existing Room ID.
2. The user establishes a Socket.IO connection with the backend server.
3. The server associates the user with the requested coding room.
4. Other users can join the same room using the same Room ID.
5. When a user modifies the code, the editor detects the change.
6. The updated code is emitted to the backend using Socket.IO.
7. The server broadcasts the update to other users in the same room.
8. Connected users receive the update and their editor reflects the new code.
9. When a user disconnects, the server notifies the remaining users.

4.3 Room-Based Communication

Each collaborative session is organized as a separate room.

This prevents code changes from one group of users from being sent to unrelated users.

For example:

- Room A may contain User 1 and User 2.
- Room B may contain User 3 and User 4.

Code changes generated inside Room A are only broadcast to users connected to Room A.

This makes the system more organized and allows multiple independent coding sessions to run simultaneously.

4.4 Future Multi-Language Execution

The current version of the project focuses primarily on real-time collaborative editing.

A future extension of the platform will allow users to select programming languages such as:

- C++
- Java
- Python
- JavaScript
- C

The user will be able to write code, provide input, execute the program, and view output or compilation errors.

Because executing arbitrary user code directly on the application server can create security risks, code execution will be handled through a secure sandbox or external code-execution service.

The future execution pipeline will follow:

Code Editor → **Execution API** / **Secure Sandbox** → **Compiler** / **Runtime** → **Output**

5 Tech Stack

The project uses modern web technologies suitable for real-time applications.

- **React:** Used to build the frontend user interface and manage application components.
- **CodeMirror:** Used to provide an interactive code editing environment.
- **Socket.IO:** Used for real-time, bidirectional communication between clients and the server.
- **Node.js:** Used to run the backend application.
- **Express.js:** Used to create and manage backend routes and server functionality.
- **UUID:** Used for generating unique Room IDs.
- **Render:** Can be used for deploying the application as a web service.

6 Evaluation Criteria

The success of the project will be evaluated using practical and measurable criteria.

- **Synchronization Accuracy:** Whether code changes made by one user are correctly reflected for other users in the same room.
- **Real-Time Response:** The delay between a code change made by one user and the update received by another user.
- **Room Isolation:** Verification that users in different rooms do not receive each other's code updates.
- **Connection Handling:** Correct handling of users joining and leaving a room.
- **Concurrent Users:** The ability of the application to support multiple users within a shared coding room.
- **Usability:** Whether users can easily create a room, join a room, and begin collaborating.

A basic experiment can be performed by opening the same Room ID on multiple browsers or devices and checking whether changes made by one user are reflected correctly for the others.

7 Scalability

The architecture is designed around independent real-time rooms.

As the number of users increases, the application can support multiple collaborative sessions by maintaining separate room connections.

Potential scalability improvements include:

- Deploying multiple backend instances.
- Using a distributed Socket.IO adapter for communication between server instances.
- Adding persistent storage for projects and code files.
- Using a database to manage users and saved projects.
- Separating the code execution service from the main collaboration server.

This approach would allow the project to evolve from a small collaborative editor into a larger collaborative coding platform.

8 Resources / Tooling

The project is designed using widely available and mostly open-source technologies.

- React and JavaScript for frontend development.
- CodeMirror for the coding interface.
- Node.js and Express for backend development.
- Socket.IO for real-time communication.
- GitHub for version control and project management.
- Render or another cloud platform for deployment.
- A secure execution API or sandbox for future multi-language code execution.

The use of open-source technologies allows the project to be developed without significant infrastructure costs.

9 Project Scope and Deliverables

9.1 Initial Deliverable

The first version of the project will include:

- Home page for entering a username and Room ID.
- Ability to create a new unique coding room.
- Ability to join an existing coding room.
- Real-time collaborative code editing.
- Display of connected users.
- Room-based communication using Socket.IO.
- User connection and disconnection handling.

9.2 Subsequent Deliverables

Future versions of the project may include:

- Multiple programming language support.
- Syntax highlighting based on selected language.
- Run code functionality.
- Custom input and output terminal.
- Compilation and runtime error display.
- Execution time information.
- User authentication.
- Saving and loading projects.
- Multi-file project support.
- Live cursor tracking.
- Integrated chat for collaborators.

10 Risks and Mitigations

- **Synchronization conflicts:** Multiple users may modify the code at nearly the same time. The initial version relies on event-based synchronization, while more advanced versions can introduce operational transformation or CRDT-based collaboration.
- **Network latency:** Slow or unstable internet connections may delay updates. Socket.IO provides reconnection support to improve reliability.
- **User disconnection:** A user may unexpectedly lose connection. The application handles disconnect events and updates the active user list.
- **Unauthorized room access:** Future versions can introduce authentication, private rooms, and permission-based access.
- **Security of code execution:** Running arbitrary user code directly on the main server can be dangerous. Future execution features will use a secure sandbox or isolated execution service.

11 Timeline

Phase	Focus
Weeks 1–2	Requirement analysis, project setup, React frontend, and room creation interface.
Weeks 3–4	CodeMirror integration and basic code editing functionality.
Weeks 5–6	Socket.IO integration and real-time synchronization between users.
Weeks 7–8	Room management, connected users, and join/disconnect handling.
Weeks 9–10	Testing with multiple users, UI improvements, and deployment.
Weeks 11–12	Future feature prototype including language selection and secure code execution integration.

12 Summary

The Real-Time Collaborative Code Editor is designed to simplify collaborative programming by providing a shared environment where multiple users can work on the same code simultaneously.

The platform combines React and CodeMirror for the frontend coding experience with Node.js, Express, and Socket.IO for real-time communication. Users can create or join coding rooms, collaborate with other users, and see code changes as they happen.

The current project focuses on solving the core challenge of real-time code synchronization. Future development will extend the platform with multi-language support, secure code execution, project storage, authentication, and additional collaborative features.

Ultimately, the project aims to evolve from a basic real-time editor into a practical collaborative coding environment for students, developers, and teams.